Attacking Passwords

"Because 'Password123' Was Never Meant to Last"

1	Introduction to Password Storage
2	Brute Forcing
3	Hash Cracking

Attacking Passwords

"Because 'Password123' Was Never Meant to Last"

1	Introduction to Password Storage
2	Brute Forcing
3	Hash Cracking

2 Introduction to Password Storage

- you develop a service, where users can create accounts and login using their username & password
- now, you have to decide on how to store this information
- when it comes to storing passwords, there is a lot of things you can do wrong
- in this section, we will work our way through different methods of storage, looking at the advantages and drawbacks of each

2 Introduction to Password Storage - Level 1: Cleartext

- maybe your first thought would be to simply store the passwords in cleartext in your DB

```
db_entry = ('john', 'treesummer55')
```

- well, if cleartext allows it to be easily readable, how about protecting its confidentiality by encrypting it?

```
cipher = ENC(KEY, 'treesummer55')
entry = ('john', cipher)
```

2 Introduction to Password Storage - Level 3: Simple Hash

- having to use (and thus store) a new key does not solve the problem, so how about something like a hash function?

```
hash = HASH('treesummer55')
entry = ('john', hash)
```

2 Introduction to Password Storage - Level 4: Salted Hashes

- to protect against pre-computation, we can assign a salt to each of the passwords, thus creating a type of uniqueness that can not be pre-computed

```
hash = HASH(SALT | 'treesummer55')
entry = ('john', SALT, hash)
```

- a salt seems to be a good addition, but we can make it even more difficult by using functions and techniques that make it much slower to hash / crack

```
h1 = HASH(SALT | 'treesummer55')
h3 = HASH(hash1)
…
hN = hash_function(hN-1)
entry = ('john', SALT, hN)
```

- to protect against pre-computation, we can assign a salt to each of the passwords, thus creating a type of uniqueness that can not be pre-computed

```
MEM = allocate_memory(M)          # e.g., 128 MB

# Fill the memory with pseudorandom blocks based on salt + password
for i in range(0, M):
    MEM[i] = HASH(SALT | password | i)

# Mix the memory repeatedly
for round in range(0, T):         # T = time cost
    for i in range(0, M):
        MEM[i] = HASH(MEM[i] | MEM[random_index])

# Final output is derived from the whole memory
final_hash = HASH(MEM[0] | MEM[1] | ... | MEM[M-1])

db_entry = ('john', SALT, final_hash)
```

2 Introduction to Password Storage - Bonus: Pepper

- while hashing, salting, and key-stretching all make password-cracking harder, none of them protect against the fact that if the database is leaked, all stored password hashes are exposed
- this is the key limitation of every hash-only defense
- a pepper is a secret stored outside the database

```
PEPPER = secret_value_stored_in_HSM_or_env     # not in DB

input = SALT | 'treesummer55' | PEPPER

hash = KDF(input)

db_entry = ('john', SALT, hash)
```

Attacking Passwords

"Because 'Password123' Was Never Meant to Last"

1	Introduction to Password Storage
2	Brute Forcing
3	Hash Cracking

3 Brute Forcing

- brute forcing describes the trial and error processes of cracking passwords, identifying login credentials encryption keys
- in this section, we will focus on remote password attacks, applies on
  - network services (ssh, ftp, ...)
  - web applications

3 Brute Forcing

- there are a variety of tools for brute forcing, such as
  - Burp Suite intruder
  - Medusa
  - NetExec (successor of CrackMapExec)
  - Hydra
- we will focus on Hydra, which can be natively found on Kali Linux
- the Hydra utility allows for the brute forcing of more than 50 different protocols, such as
  - FTP
  - HTTP
  - SSH
  - ...

3 Brute Forcing - Hydra Command

- to use hydra, you will generally require the following information
  - a target (IP or host name)
  - the type service to be brute forced (`ssh`, `ftp`, `smb`, ...)
  - either a single username (if required) or a file containing a list of usernames to be brute forced
  - either a single password (if required) or a file containing a list of usernames to be brute forced

3 Brute Forcing - Hydra Command

```
[user@kali ~]$ hydra -L <username_files> -P
<password_file> <target> <service>

[user@kali ~]$ hydra -l <username> -P <password_file>
<target> <service>

[user@kali ~]$ hydra -L <username_files> -p <password>
<target> <service>

[user@kali ~]$ hydra -l <username> -p <password>
<target> <service>
```

3 Brute Forcing - Hydra Command

Switch	Description
`-o <out_file>`	save output in a file
`-f`	stop, as soon as the first valid credentials were found
`-s <port>`	specify target port, if it deviates from the default port
`-M <targets_file>`	to specify multiple targets
`-R`	restore a previously cancelled/crashed brute forcing session
`-I`	ignore a pre-existing restore file (i.e., skip 10s wait)

3 Brute Forcing - Wordlists

- as it is the case for fuzzing, the choice of the wordlist used for brute forcing is crucial
- if possible, it should adapt to the situation at hand
- recommendations:
  - `/usr/share/wordlists/rockyou.txt`
  - Seclists
    - `/usr/share/wordlists/seclists/Usernames/`
    - `/usr/share/wordlists/seclists/Passwords/`

3 Brute Forcing - Web Login Forms

- Hydra also allows for brute forcing web application login forms
  - Whether an attempt was successful or not depends on a certain signal string that we need to provide

```
[user@kali ~]$ hydra -l admin -P
/usr/share/wordlists/rockyou.txt 10.10.10.43
http-post-form
"/department/login.php:username=admin&password=^PAS
S^:Invalid Password!"
```

3 Brute Forcing - Usefulness

- trying to just brute force random users on random services will most often not be the most efficient approach and seldomly yield results
- instead, brute forcing becomes more useful if
  - we already found a username / password
  - we gained knowledge, that a user's password is especially vulnerable
  - we learned about an insecure password policy

Attacking Passwords

"Because 'Password123' Was Never Meant to Last"

1	Introduction to Password Storage
2	Brute Forcing
3	Hash Cracking

4 Hash Cracking

- when gaining access to system files during the privilege escalation phase or the post-exploitation phase, we might encounter password hashes
  - any type of mechanism that uses password based authentication, needs to store the associated password hashes somewhere
  - examples:
    - system account on linux: `/etc/shadow`
    - system accounts on Windows: `%SystemRoot%/system32/config/SAM`
    - web application accounts: e.g., `/var/www/html/config.php`

4 Hash Cracking

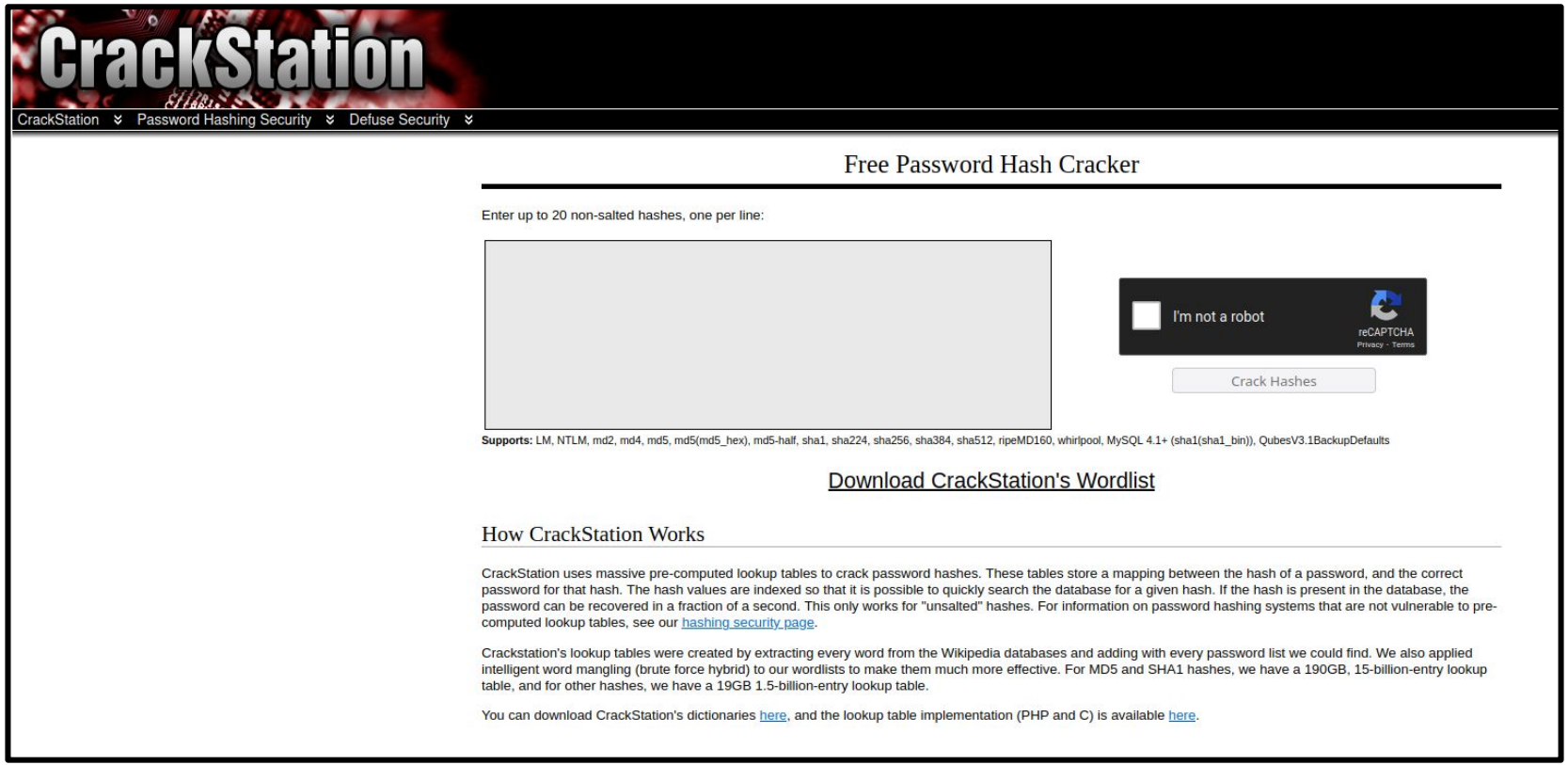
- hash cracking refers to the process of identifying the string that was used to generate a certain hash
- hash cracking tools will try to achieve this goal by
  - taking a list of potential strings,
  - hashing them with hash function in question, and
  - comparing them to the original hash
- hash cracking tool can generally be divided into two categories
  - online hash crackers
  - hash cracking software

4 Hash Cracking - Online Hash Cracking

- online hash crackers usually work by comparing entered hashes to a list of precomputed hash-plaintext-pairs
- <u>advantages</u>
  - no computation necessary → immediate results
- <u>disadvantages</u>
  - limited support
    - only certain hashing functions considered
    - no salt-based hashing functions considered
  - exposure to third parties of potentially sensitive data

4 Hash Cracking

- one well-known online cracker ist [crackstation]



4 Hash Cracking - Hash Cracking Software

- conversely, hash cracking software functions by converting a list of plaintext strings into hashes
- advantages
  - more flexibility
  - no sensitive information exposure
- disadvantages
  - requires computation time

4 Hash Cracking - Hash Cracking Software

- when it comes to hash cracking utilities, the two well most known implementations are
  - John the Ripper (JtR)
  - hashcat
- both of which are shipped with a basic Kali Linux installation
- we will be demonstrating the basic process of hash cracking with JtR

- when it comes to cracking passwords, JtR offers different modes for how to approach the problem
  - **wordlist mode**
    - input is a wordlist file, where each line contains one candidate plaintext string
    - JtR will hash all the strings, top to bottom, and compare the resulting hashes with the goal hash
    - can be used in conjunction with rules, i.e., certain modifications and/or filters to be made to the list of strings

```
[user@kali ~]$ john --wordlist=/usr/…/rockyou.txt hashes.txt
```

- when it comes to cracking passwords, JtR offers different modes for how to approach the problem
  - **single crack mode**
    - similar to wordlist mode
    - however, the input wordlist is not explicitly supplied, but rather it will create its own by using
      - the values of GECOS fields (e.g., the information contained in an entry of an `/etc/passwd` file),
      - applying some managing rules on these values

```
[user@kali ~]$ john --single hashes.txt
```

- when it comes to cracking passwords, JtR offers different modes for how to approach the problem
  - **incremental mode**
    - works by brute forcing the original string, i.e., by generating all possible plaintexts and hashing them
    - input is the maximal length of the generated plaintext and the set of characters to use
    - in theory, guaranteed to succeed, but may take hundreds of years (at least until quantum computers are viable)

```
[user@kali ~]$ john --incremental
--incremental_charcount=<N> hashes.txt
```

4 Hash Cracking - JtR - Hash Format

- JtR will automatically try and identify the hash format used
  - this will often work, but sometimes you might want to explicitly state the hash format to be used
- given that you know the hash format, you can use the following syntax to specify it

```
[user@kali ~]$ john --format=[format] hashes.txt
```

- for a useful list of available formats, see here

4 Hash Cracking - JtR - Hash Format

- to identify an unknown hash format, consider the following options
  - hash-identifier
  - hashcat wiki
  - dcode

4 Hash Cracking - JtR - Results

- whenever JtR was able to crack a password, it will be displayed in the terminal
  - subsequent attempts to crack the same hash or hash file will result in JtR informing you, that the specific hashes were already cracked
  - to display a list of the cracked hashes of an associated hash file, use the following

```
[user@kali ~]$ john --show <hash_file.txt>
```

4 Hash Cracking - JtR - Hash Extraction

- while files like the `/etc/shadow` file contain explicit hashes, other password-based authentication mechanisms do no have an dedicated and separate hash file
- for example:
  - password-protected ZIP files
  - password-protected PDF files
  - password-protected private SSH keys
  - password-protected password-manager database files
  - ...
- to actually crack the associated hash, we will first have to extract and isolate it

4 Hash Cracking - JtR - Hash Extraction

- JtR comes with a suite of utilities for exactly this purpose

```
[user@kali ~]$ zip2john file.zip > extracted_zip_hash

[user@kali ~]$ pdf2john file.pdf > extracted_pdf_hash

[user@kali ~]$ ssh2john id_rsa > extracted_ssh_hash

...
```

(Re)Sources

- https://portswigger.net/burp/documentation/desktop/tools/intruder
- https://github.com/jmk-foofus/medusa
- https://www.netexec.wiki/
- https://github.com/byt3bl33d3r/CrackMapExec
- https://github.com/vanhauser-thc/thc-hydra
- https://crackstation.net/
- https://www.openwall.com/john/
- https://hashcat.net/hashcat/
- https://pentestmonkey.net/cheat-sheet/john-the-ripper-hash-formats
- https://hashcat.net/wiki/doku.php?id=example_hashes
- https://www.dcode.fr/hash-identifier
- https://infinitelogins.com/2020/02/22/how-to-brute-force-websites-using-hydra/